

ElectronBot — Entendimento Profundo do Projeto

Gerado por análise multi-agente (37 agentes: 5 exploradores por pasta × 6 pastas + 6 sintetizadores + 1 síntese global). Somente leitura — nada foi modificado.

ElectronBot é o robô-companheiro de mesa open-source do **peng-zhihui** — uma figura do tamanho da palma da mão, estilo WALL-E/EVE, com um **rosto de LCD redondo 240×240**, uma **câmera USB** embarcada e **6 graus de liberdade** (2 braços × roll+pitch, mais pescoço e cintura).

A ideia arquitetural mais importante: o robô em si é "burro". Toda a inteligência — renderização de emoji/expressão, animação, planejamento de movimento — roda num **PC host**. O firmware STM32 é apenas uma ponte fina que joga pixels no LCD e repassa ângulos para os servos. O rosto é vídeo; o movimento do corpo são seis floats; ambos viajam por **um único cabo USB**.

Fluxo de dados ponta a ponta

1. **Unity Studio (PC, C#)** — autora/reproduz keyframes 3D de pose+expressão; calcula 6 ângulos de junta e escolhe imagem/vídeo de rosto por frame.
2. **UnityBridge.dll (C++)** — Unity faz P/Invoke de 4 funções `Native_*`; empurra a imagem + 6 setpoints e retorna juntas medidas + frame da webcam (invertido).
3. **ElectronBotSDK-LowLevel.dll (C++)** — a classe única (`ElectronLowLevel`) que detém o protocolo de fio. Cada `Sync()` redimensiona o rosto para 240×240 RGB888 (172.800 bytes) e o transmite enquanto lê telemetria.
4. **USBInterface.dll + libusb-win32 (BotDriver)** — transporte USB bulk cru (NÃO é porta COM virtual).
5. **ElectronBot-fw no STM32F405 (placa da cabeça)** — recebe o frame num ping-pong buffer, faz DMA para o LCD GC9A01 via SPI1 e atua como I2C master dos 6 servos.
6. **ServoDrive-fw no STM32F042 (um por junta)** — escravos I2C; cada um roda um loop PID de posição a 200 Hz acionando um motor DC escovado (ponte-H), lendo um potenciômetro via ADC para feedback de ângulo absoluto.

O **mesmo canal de 32 bytes** que leva os 6 setpoints de saída também retorna os 6 ângulos medidos a cada frame — vídeo de rosto e movimento 6-DOF ficam sincronizados, e o PC pode ler de volta o robô posicionado à mão (modo "physical-priority").

1. Hardware — 5 projetos Altium (2 camadas, prontos p/ fabricação)

Topologia = **árvore USB + um barramento I2C compartilhado.**

Placa	MCU	Papel
BaseConnector	nenhum	Entrada USB-C (sink) na base giratória. Liga ao corpo por flex FFC 8 vias 0,5 mm (a cintura gira).
SensorBoard	nenhum	"Coração elétrico" do torso: hub USB HS8836A 4 portas, sensor de gesto PAJ7620U2 , IMU MPU6050 , 5 conectores de servo SH1.0-4P. Tudo num I2C compartilhado.
ElectronBot (cabeça)	STM32F405RGT6 (M4F, 168 MHz, 1MB)	O cérebro. LCD redondo GC9A01 240×240 via SPI, USB High-Speed via PHY externo USB3300 (por isso <i>tem</i> de ser F4), ponte CP2102 UART, slot microSD. I2C master.
ServoDrive	STM32F042F6P6 (M0)	Uma placa dentro de cada servo RC modificado + driver ponte-H DC escovado FM116B . Escravo I2C, lê pot via ADC, roda PID.
ServoDrive-DK	STM32F042	Gêmeo de debug eletricamente idêntico ao ServoDrive, com headers plugáveis para flashar/depurar na bancada.

⚠ **Não confundir** ServoDrive (STM32F0 + DC escovado) com a placa da cabeça (STM32F4). **Não há FOC/BLDC** em lugar nenhum. SensorBoard e BaseConnector não têm firmware porque não têm MCU.

2. Firmware — 3 projetos STM32 (sistema master/slave)

ElectronBot-fw (cabeça, STM32F405, M4F) — C gerado pelo CubeMX/HAL em `Core/` + C++17 escrito à mão em `UserApp/` / `Bsp/`. Boota HAL + FreeRTOS, mas o app real é um **super-loop bare-metal** em `Main()` (FreeRTOS é só um launcher). Espera 2 s os servos bootarem (`HAL_Delay(2000)`), evita travar o I2C). Por frame: 4 iterações enviando pacote IN de 32 bytes (com os 6 ângulos medidos), busy-wait pela imagem OUT, DMA de uma faixa LCD 60×240×3 = 43.200 bytes (RAMWR 0x2C na faixa 0, RAM-continue 0x3C depois), e após 4 faixas comanda as 6 juntas por I2C. DMA-SPI de uma faixa sobrepõe a recepção USB da próxima via **ping-pong buffer**. **I2C master** nos endereços pares 2,4,6,8,10,12 (pescoço, L-roll, L-pitch, R-roll, R-pitch, cintura).

ServoDrive-fw / ServoDrive-fw-II (junta, STM32F042, M0) — **escravos I2C**. Loop de 200 Hz na ISR do TIM14 lê **potenciômetro** no ADC1_IN4/PA4 (NÃO encoder magnético), mapeia contagens calibradas (250..3000 → 0..180°), roda PID de posição (o "DCE": $k_p \cdot \text{err} + k_i \cdot \int + k_v \cdot \dot{\text{vel}} + k_d \cdot \Delta$, anti-

windup) e aciona motor DC escovado como ponte-H sign-magnitude no TIM3 (~48 kHz). Opcodes I2C: 0x01 set ângulo, 0x02 velocidade, 0x03 torque, 0x11/0x12 ler ângulo/velocidade, 0x21 set id, 0x22–0x25 ganhos PID, 0x26 limite torque, 0x27 pos inicial, 0xff enable/disable; a resposta sempre ecoa o ângulo atual. Config persiste em flash (EEPROM emulada).

As **duas variantes de servo têm lógica byte-a-byte idêntica** (motor.cpp, PID DCE, handler I2C). Diferem só em: (a) camada de periférico — `ServoDrive-fw` = HAL; `ServoDrive-fw-ll` = LL (menor/rápido, **troca de endereço I2C ao vivo sem reboot**, adiciona alvo F030); (b) toolchain (CMake/CubeIDE vs PlatformIO).

3. Software — Stack host (PC) em camadas

- **Driver USB (base):** `_Tools/BotDriver` instala **libusb-win32** → robô enumera como **dispositivo bulk cru, não porta COM**. VID **0x1001** / PID **0x8023** (app); 0x0483/0x5740 (bootloader).
- **Transporte:** `USBInterface.dll` (C-ABI sobre libusb0). Endpoints **EP1_OUT 0x01** (host→device), **EP1_IN 0x81** (device→host).
- **Núcleo — ElectronBotSDK-LowLevel** : a classe `ElectronLowLevel` que é o link. Protocolo por `Sync()` :
- `SetImageSrc` → 240×240, BGRA→RGB = **172.800 bytes** num ping-pong TX.
- `Sync()` envia o frame em **4 chunks ritmados pelo MCU**. Por chunk: (1) **lê 32 bytes em EP1_IN** (handshake + telemetria dos 6 ângulos), (2) escreve **84 × 512 B = 43.008 B** de imagem, (3) escreve **cauda de 224 B** (192 imagem + 32 extra-data). $4 \times (43.008 + 192) = 172.800$ = um frame.
- **Bloco extra de 32 bytes:** `byte[0]` = flag enable, `bytes[1..24]` = 6 floats little-endian (setpoints). O **pacote curto de 224 B é o delimitador de fim de frame** que o firmware usa para virar o ping-pong.
- **Camada média:**
- **Player** — toca imagens ou MP4/MOV via OpenCV. ⚠ `SetPose` / `GetPose` são **stubs vazios** (header LowLevel antigo sem `SetJointAngles`) → **o Player só controla a tela, não os servos**.
- **UnityBridge** — plugin `extern "C"` com 4 entradas (`Native_OnInit/OnFixUpdate/OnKeyFrameChange/OnExit`); `OnFixUpdate` é não-bloqueante (thread destacada).
- **Front-end 1 — Unity/ElectronBot-Studio** (Unity 2020.1.6f1, C#): rig 6-DOF na tela via sliders; P/Invoke das 4 `Native_*` por `FixedUpdate` (~20 Hz). 3 modos: **0 = model-priority** (PC comanda), **1 = disabled**, **2 = physical-priority** (lê robô posicionado à mão). Autoria de keyframes guarda {6 ângulos, backlight, mídia}. `Unity/Release` traz players Windows/Linux v0.0.1.

- **Front-end 2** — `_Tools/AHK-ExpansionPack` (AutoHotkey v1): build separado do SDK (OpenCV 4.5.5). Limites de junta: cabeça ± 15 , cintura ± 90 , abrir braço ± 30 , levantar braço ± 180 . 8 demos (controle WeChat, espelho de rosto, voz, etc.).
- **Calibração** — `_Tools/ServoToolKit` : GUI Qt5 via **libusb-1.0**. **Requer flashar seu próprio firmware** (`ElectronBot-fw-servo-toolkit.hex`) na cabeça, substituindo o app temporariamente; conecte **um servo por vez** quando endereços não estão setados.

⚠ **Version drift:** CMake aponta OpenCV 3.4.8 (`opencv_world348d` , debug) mas DLLs entregues são 4.5.5; CMakeLists têm caminhos fixos `D:\Libraries\Cpp\...` .

4.CAD-Model — Modelo mecânico + assets do rosto

Pasta só de entregáveis (sem código). Dois STEP na raiz: **ElectronBot.step** (~31,7 MB, gêmeo digital eletromecânico completo — cascas, engrenagens, rolamentos, parafusos M2, microserves e geometria das PCBs) e **Box.step** (caixa imprimível em FDM do fim do vídeo, NÃO a base elétrica). Sólidos B-rep/NURBS em mm, exportados de **Fusion 360** (robô) e **Rhino3D** (caixa); fonte `.f3d` editável só num link A360 externo.

Emoji/ = biblioteca de máquina de estados de animação: 6 emoções (desdém, animado, apavorado, raiva, triste, estático). Cada emoção segue gramática **enter** → **loop** → **return** (`_1进入姿势` , `_2可循环动作` , `_3回正` , `_动作演示`) p/ emendar expressões; idle adiciona piscadas e olhares aleatórios.

⚠ **Fato corrigido:** os cliques de emoji são **900×900 H.264, NÃO 240×240**. O downscale acontece **no PC** (OpenCV `cv::resize` + RGB888), nunca no MCU. Renomeie arquivos p/ ASCII antes de usar — o Studio rejeita caminhos em chinês.

5.Docs — Loja de referência (sem markdown)

- **Datasheets/** — 11 PDFs = BOM ao nível de chip (GC9A01, PAJ7620U2, USB3300, hubs GL850G/SL2.1A/FE8.1/FM116B, STMs). ⚠ O F405 (principal) e o F042 (servo) reais **não têm datasheet aqui** — buscar na ST.
- **Images/** — `robot9.png` = **tabela autoritativa do protocolo I2C dos servos**; `robot7.jpg` = diagrama do stack de software; `robot4.jpg` = topologia USB.
- **_LargeFiles/** — **ação crítica de build:** seguindo `_path.txt` , descompacte `msyh.zip` → `Assets/Materials/Fonts/msyh.asset` e `opencv_world348d.zip` → `Assets/Plugins/opencv_world348d.dll` dentro do projeto Unity (excluídos pelo `.gitignore` por causa do limite de 100 MB do GitHub).

A documentação narrativa real está no `README.md` (**chinês, autoritativo**) / `enREADME.md` na raiz, não em 5.Docs.

6.Tests — TestDisplayUSB (bring-up turnkey)

Teste de integração canônico que prova o robô ponta a ponta (PC → USB → LCD, com caminho de servo dormente). Traz fonte de firmware + fonte do SDK host + **binários prontos** em `_Released/`. O `ElectronBot-fw` aqui é um snapshot do firmware canônico de `2.Firmware`. VID/PID batem nos dois lados (0x1001/0x8023). Mesma matemática de frame: $4 \times (84 \times 512 \text{ B} + 224 \text{ B}) = 172.800 \text{ B}$.

Por onde começar (build, flash, run)

Caminho mais rápido — `6.Tests/TestDisplayUSB` : 1. **Instale o driver USB** primeiro:

`3.Software/_Tools/BotDriver` (pré-requisito de tudo). (Pode ser preciso vincular o driver ao VID/PID do app via Zadig.) 2. **Flashe o firmware:** `_Released/STM32F4 Firmware/ElectronBot-fw.hex` via **ST-Link/SWD** (OpenOCD, stm32f4x). 3. **Rode o demo:** `_Released/Windows EXE/Sample.exe`. Sucesso = console imprime **"Robot connected!"** e `happy.mp4` anima no rosto.

Para desenvolver: - **Firmware:** `2.Firmware/ElectronBot-fw` (CMake/arm-none-eabi-gcc ou `.ioc` no CubeMX) p/ cabeça; `ServoDrive-fw` (CMake) ou `ServoDrive-fw-11` (PlatformIO) p/ juntas. Flashe servo no **ServoDrive-DK**, depois grave no ServoDrive de produção. - **SDK + Unity:** build de `3.Software/SDK` (CMake ≥3.21, MSVC x64). **Antes**, extraia `5.Docs/_LargeFiles` conforme `_path.txt`. Recrie os caminhos `D:\Libraries\Cpp\...` (OpenCV está em `SDK/_Libs/OpenCV_lib.zip`). - **Calibrar servos:** flashe o hex do `ServoToolKit`, ajuste ID/PID/limites de cada servo um por vez, re-flashe o firmware normal.

O único arquivo para entender o sistema inteiro:

`3.Software/SDK/ElectronBotSDK-LowLevel/electron_low_level.cpp` — define todo o protocolo de fio PC↔robô; todo o resto é wrapper em volta dele.